

Go2 机器狗 Waypoint 避障算法教程

基于 ROS 2 Bridge、自研路点跟踪与 Unitree 避障接口的当前项目说明

RSSI/IMU Go2 项目

2026 年 6 月

目录

第一章 总体视角：当前避障算法到底是什么	3
1.1 一句话概括	3
1.2 为什么不是“完整 Nav2 避障”	3
1.3 当前算法总图	4
1.4 模块分工	4
第二章 模块一：waypoint 路线如何表达和处理	6
2.1 relative_waypoints 的格式	6
2.2 L 型路线示例	6
2.3 launch 如何解析 waypoint	7
2.4 相对路点如何变成 odom 坐标	7
2.5 路点到达判定	8
第三章 模块二：自研 waypoint follower 如何算速度	9
3.1 输入输出	9
3.2 控制律	9
3.3 接近路点时怎么处理	10
3.4 它负责什么，不负责什么	10
第四章 模块三：go2_nav2_bridge 的接口转换与安全门控	11
4.1 bridge 为什么必要	11
4.2 bridge 发布的 ROS 2 话题	11
4.3 速度命令如何进入 Unitree SDK	12
4.4 安全门控	12
4.5 点云处理	12
第五章 模块四：Unitree ObstaclesAvoidClient 的角色	14
5.1 为什么使用 obstacles_avoid	14
5.2 它和上层 follower 的关系	14
5.3 它的优点和限制	14

第六章	为什么不采用 Nav2 官方 waypoint follower	16
6.1	Nav2 waypoint follower 实际做什么	16
6.2	如果直接用 Nav2 waypoint follower, 会发生什么	16
6.3	为什么当前选择自研 follower	17
6.4	这不是否定 Nav2	17
第七章	完整 Nav2 方案什么时候值得切换	18
7.1	适合切换的场景	18
7.2	完整 Nav2 方案的链路	18
7.3	两种方案对比	19
第八章	如何运行和观察当前算法	20
8.1	编译	20
8.2	运行单段路线	20
8.3	运行 L 型路线	20
8.4	观察话题	20
第九章	调参指南	22
9.1	路线跟踪参数	22
9.2	安全参数	22
9.3	问题排查	22

第一章 总体视角：当前避障算法到底是什么

1.1 一句话概括

当前项目的避障算法不是完整 Nav2 导航栈，也不是让 Nav2 官方 waypoint follower 直接控制机器人。当前采用的是一个更适合 RSSI/IMU 实验采集的混合架构：

waypoint 路线 → 自研 odom 路点跟踪器 → /cmd_vel
→ Go2 ROS 2 bridge → Unitree ObstaclesAvoidClient
→ Go2 机载避障执行

这套设计的核心思想是：上层只规定“走哪条实验路线”，中间层根据 odom 生成速度命令，底层由宇树官方避障接口负责近距离防撞和绕障。这样可以避免在 RSSI/IMU 数据采集阶段过早陷入完整 Nav2 costmap、footprint、DWB critic、obstacle range 等大量细节调参。

当前代码主入口：

```
ros2_ws/src/go2_nav2_bridge/launch/go2_waypoint_obstacle_avoid.launch.py
```

它默认启动 go2_nav2_bridge 和 go2_waypoint_follower，并强制 bridge 使用 Unitree obstacles_avoid 运动后端。

1.2 为什么不是“完整 Nav2 避障”

完整 Nav2 避障通常是：

目标点 → planner_server → global path
→ controller_server → DWB/RPP → /cmd_vel

这要求 Nav2 自己理解环境障碍，所以必须认真配置：

- robot footprint 或 robot radius;

- local/global costmap;
- VoxelLayer 或 ObstacleLayer;
- obstacle range、raytrace range;
- inflation radius;
- controller 插件，如 DWB 或 Regulated Pure Pursuit;
- TF、odom、点云坐标系、速度/加速度限制。

而当前项目的实验目标首先是让 Go2 按可复现路线采集 RSSI/IMU 数据。对这个阶段来说，最重要的是路线可控、速度温和、日志可解释、出问题时容易定位。因此当前没有把 Nav2 controller/costmap 作为主避障链路。

1.3 当前算法总图

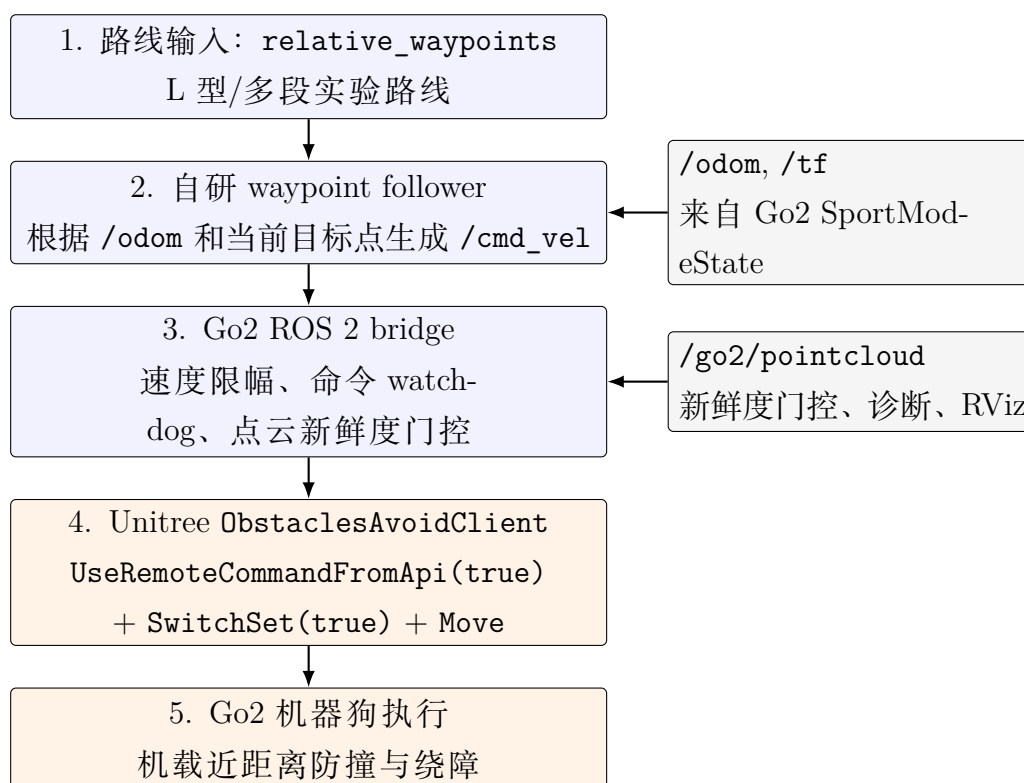


图 1.1: 当前 Go2 waypoint 避障算法的数据流和控制流。

1.4 模块分工

模块	当前是否启用	作用
go2_waypoint_follower	是	读取相对 waypoint 和 /odom，计算 /cmd_vel。它负责路线跟踪，不负责点云避障。
go2_nav2_bridge	是	连接 ROS 2 与 Unitree SDK。发布 odom/IMU/点云，订阅 /cmd_vel 并下发给 SDK。
Unitree ObstaclesAvoidClient	是	当前近距离防撞和绕障的主要执行接口。
/go2/pointcloud	是	bridge 发布点云，用于新鲜度门控、诊断、可视化；当前没有进入 Nav2 costmap 主链路。
Nav2 planner/controller/-costmap	参数保留	当前 waypoint launch 不启动。未来若需要完整 Nav2 避障，可以接入。
Nav2 官方 waypoint follower	否	当前不使用，原因见第 六 章。

表 1.1: 当前 waypoint 避障算法的模块分工。

第二章 模块一：waypoint 路线如何表达和处理

2.1 relative_waypoints 的格式

当前路线通过 launch 参数 `relative_waypoints` 给出。它是一个数字列表，每三个数是一组：

$$[x_1, y_1, yaw_1, x_2, y_2, yaw_2, \dots]$$

其中：

- x_i ：相对起点坐标系的前向距离，单位 m；
- y_i ：相对起点坐标系的左向距离，单位 m；
- yaw_i ：到达路点时希望的相对航向，单位 rad。

默认值是：

```
relative_waypoints: [3.0, 0.0, 0.0]
```

含义是：从启动位置向前走 3 m，并保持启动时的航向。

2.2 L 型路线示例

比如要实现“先前进 3 m，到拐点，左转，再前进 2 m”，可以写成：

```
ros2 launch go2_nav2_bridge go2_waypoint_obstacle_avoid.launch.py \  
  network_interface:=eth0 \  
  relative_waypoints:="[3.0, 0.0, 1.57, 3.0, 2.0, 1.57]"
```

解释如下：

- 第一个路点 `[3.0, 0.0, 1.57]`：相对起点向前 3 m，到达后朝左约 90° 。

- 第二个路点 [3.0, 2.0, 1.57]：仍以起点坐标系为参考，目标位置是前方 3 m、左侧 2 m，航向保持左转后的方向。

当前所有 waypoint 都是相对于启动时的起点坐标系，不是相对于上一个 waypoint。也就是说第二个点不是“从第一个点再走 2 m”，而是“在起点坐标系下的绝对相对坐标”。

2.3 launch 如何解析 waypoint

当前 waypoint launch 使用 `OpaqueFunction` 和 `ast.literal_eval` 解析 `relative_waypoints`。这样做比直接把字符串塞给参数更稳，因为它会检查：

- 输入必须是 list 或 tuple；
- 长度不能为 0；
- 长度必须是 3 的倍数；
- 每个元素都能转成 float。

解析完成后，launch 把数值列表传给 `go2_waypoint_follower`：

```
Node(
  package="go2_nav2_bridge",
  executable="go2_waypoint_follower",
  parameters=[
    params_file,
    {"relative_waypoints": relative_waypoints},
  ],
)
```

2.4 相对路点如何变成 odom 坐标

`go2_waypoint_follower` 第一次收到 `/odom` 时，会记录当前位姿为起点：

$$(x_0, y_0, \psi_0)$$

之后每个相对路点 (x_r, y_r, ψ_r) 转成 odom 坐标系下的目标：

$$x_g = x_0 + \cos \psi_0 \cdot x_r - \sin \psi_0 \cdot y_r$$

$$y_g = y_0 + \sin \psi_0 \cdot x_r + \cos \psi_0 \cdot y_r$$

$$\psi_g = \text{wrap}(\psi_0 + \psi_r)$$

这一步的意义很大：实验人员不用知道场地中的绝对坐标，只要按机器人启动时的朝向设计路线即可。

2.5 路点到达判定

当前有两个容差：

- `xy_tolerance_m`: 位置误差小于该值，认为到达目标位置；
- `yaw_tolerance_rad`: 航向误差小于该值，认为达到目标朝向。

只有位置和航向都满足要求，`follower` 才会切换到下一个 `waypoint`。全部 `waypoint` 完成后，节点发布零速度。

第三章 模块二：自研 waypoint follower 如何算速度

3.1 输入输出

方向	名称	作用
输入	/odom	当前机器人位置、姿态、速度。
输入	relative_waypoints	需要跟踪的路线。
输出	/cmd_vel	发给 bridge 的速度命令。
参数	k_xy, k_yaw	位置和角度比例控制增益。
参数	max_vx_mps, max_vy_mps, max_wz_radps	follower 侧速度上限。

3.2 控制律

对当前目标点，follower 计算：

$$d = \sqrt{(x_g - x)^2 + (y_g - y)^2}$$

$$\theta_e = \text{wrap}(\text{atan2}(y_g - y, x_g - x) - \psi)$$

其中 d 是距离误差， θ_e 是机器人当前朝向和目标方向之间的夹角。
当前速度计算为：

$$v_x = \text{clamp}(k_{xy} d \max(\cos \theta_e, 0), 0, v_{x,\max})$$

$$v_y = \text{clamp}(k_{xy} d \sin \theta_e, -v_{y,\max}, v_{y,\max})$$

$$\omega_z = \text{clamp}(k_{yaw} \theta_e, -\omega_{\max}, \omega_{\max})$$

这套控制律有几个直观效果：

- 目标越远，前进速度越大；
- 目标越偏左，横向速度和角速度越偏左；
- 如果目标在身后， $\cos \theta_e$ 可能为负，此时前向速度被压到 0，避免倒着硬走；
- 速度最终被上限裁剪，保证实验阶段运动温和。

3.3 接近路点时怎么处理

如果位置已经进入 `xy_tolerance_m`，但 `yaw` 还没达到 `yaw_tolerance_rad`，`follower` 不再平移，只发布角速度：

$$v_x = 0, \quad v_y = 0, \quad \omega_z = \text{clamp}(k_{yaw}(\psi_g - \psi), -\omega_{\max}, \omega_{\max})$$

这样 L 型路线在拐点处会先调整方向，再进入下一段运动。

3.4 它负责什么，不负责什么

go2_waypoint_follower 负责	go2_waypoint_follower 不负责
解析路点列表；记录起点；将相对路点转换到 odom；按顺序跟踪 waypoint；生成平滑、低速、可解释的 <code>/cmd_vel</code> 。	读取点云；建立 costmap；对障碍物做全局重规划；像 DWB 一样对多组候选速度评分；直接调用 Unitree SDK。

表 3.2: 自研 waypoint follower 的职责边界。

避障执行发生在后面的 `bridge` 和 `Unitree ObstaclesAvoidClient`。

第四章 模块三：go2_nav2_bridge 的接口转换与安全门控

4.1 bridge 为什么必要

ROS 2 控制器输出的是 `/cmd_vel`，Nav2 和 RViz 需要的是 `/odom`、TF、点云等标准话题。但 Go2 侧原生接口来自 Unitree SDK/DDS，例如：

- `rt/sportmodestate`：运动状态；
- `rt/utlidar/cloud`：点云；
- `SportClient::Move`：普通速度控制；
- `ObstaclesAvoidClient::Move`：带官方避障能力的速度控制。

`go2_nav2_bridge` 是这两套接口之间的转换层。

4.2 bridge 发布的 ROS 2 话题

话题	类型	来源与作用
<code>/odom</code>	<code>Odometry</code>	来自 Go2 位置、速度和姿态。follower 依赖它闭环。
<code>/tf</code>	<code>TF</code>	发布 <code>odom -> base_link</code> 。
<code>/go2/imu</code>	<code>Imu</code>	来自 Go2 IMU，供 RSSI/IMU 融合使用。
<code>/go2/pointcloud</code>	<code>PointCloud2</code>	Unitree 点云经坐标修正和自车点过滤后发布。
<code>/go2/path_s</code>	<code>Float32</code>	相邻位置增量累加得到的路径长度估计。

表 4.1: bridge 发布的主要 ROS 2 话题。

4.3 速度命令如何进入 Unitree SDK

follower 发布 `/cmd_vel`, bridge 的 `onTwist()` 回调会调用 `sendVelocity(vx,vy,wz)`。这个函数做四件事:

1. 把 NaN 或无穷值变成 0;
2. 根据 `max_vx_mps`、`max_vy_mps`、`max_wz_radps` 限幅;
3. 如果开启点云新鲜度门控, 则检查 `/go2/pointcloud` 是否超时;
4. 根据 `motion_client` 选择 `ObstaclesAvoidClient::Move` 或 `SportClient::Move`。

当前 waypoint launch 强制使用:

```
motion_client: obstacles_avoid
obstacle_avoid_switch_on_start: true
obstacle_avoid_remote_api_on_start: true
```

因此最终运动调用是:

```
obstacle_client_->Move(vx, vy, wz)
```

4.4 安全门控

门控	作用
速度限幅	防止上游 follower 或其他节点发出过大的速度命令。
非有限值处理	避免 NaN/Inf 进入 SDK。
点云新鲜度检查	<code>require_fresh_point_cloud_for_motion=true</code> 时, 非零速度必须建立在最近有点云更新的基础上。
命令 watchdog	如果超过 <code>cmd_timeout_sec</code> 没收到新 <code>/cmd_vel</code> , bridge 会发停止。
析构停止	bridge 退出时调用 <code>sendStop()</code> , 尽量保证机器人不残留速度。

4.5 点云处理

当前 bridge 对 Unitree 点云做三步处理:

1. 转换 DDS 点云到 ROS 2 `PointCloud2`;

2. 根据 `point_cloud_xyz_offset` 和 `point_cloud_rpy_offset` 修正坐标;
3. 根据 `self_filter_boxes` 把自车附近点设为 NaN。

这些处理对两件事有用:

- 让 RViz 和诊断脚本看到更接近 `base_link` 坐标系的点云;
- 让点云新鲜度门控和未来 Nav2 costmap 接入更可靠。

但当前主避障链路里, `/go2/pointcloud` 没有进入 Nav2 VoxelLayer/DWB。它更多是安全门控、诊断和未来扩展的基础。

第五章 模块四：Unitree ObstaclesAvoidClient 的角色

5.1 为什么使用 obstacles_avoid

Go2 可以用普通运动接口发送速度，也可以用官方避障运动接口发送速度。当前 waypoint 避障算法选择后者，因为实验阶段不希望上层自己承担全部近距离避障细节。

启动时 bridge 会做：

```
obstacle_client_->UseRemoteCommandFromApi(true);  
obstacle_client_->SwitchSet(true);
```

含义是：

- 允许 API 远程命令控制；
- 打开官方避障开关；
- 之后的速度命令通过 ObstaclesAvoidClient::Move(vx,vy,wz) 发送。

5.2 它和上层 follower 的关系

follower 输出的是“期望速度”，不是最终真实运动轨迹。Unitree obstacles_avoid 在执行时可能因为近距离障碍调整实际运动，甚至拒绝某些危险速度。

因此当前系统可以理解为两层：

- 上层路线层：我希望去哪个 waypoint；
- 下层安全执行层：在不撞障碍的前提下尽量执行这个速度。

5.3 它的优点和限制

类型	说明
----	----

优点	避免在早期实验中大量调 Nav2 costmap/footprint/obstacle range。
优点	直接使用 Go2 官方运动能力，和机载感知/控制结合更紧。
优点	上层路线控制更简单，RSSI/IMU 数据采集更容易复现。
限制	官方避障是黑盒，无法像 Nav2 DWB critic 那样解释每个障碍点如何影响速度评分。
限制	如果官方避障改变了实际轨迹，odom 轨迹会偏离理想 waypoint 路线。
限制	它不是全局规划器。如果通道完全堵住，它不一定能规划绕远路。

第六章 为什么不采用 Nav2 官方 waypoint follower

6.1 Nav2 waypoint follower 实际做什么

Nav2 的 `nav2_waypoint_follower` 名字里有 follower，但它不是底层速度控制器。它更像一个任务调度器：

waypoint 列表 → `nav2_waypoint_follower` → `NavigateToPose`
→ 完整 Nav2 导航链路

真正把目标点变成 `/cmd_vel` 的不是 waypoint follower，而是 Nav2 的 planner、controller、costmap、行为树等模块。

6.2 如果直接用 Nav2 waypoint follower，会发生什么

链路会变成：

`nav2_waypoint_follower` → `bt_navigator` → `planner_server`
→ `controller_server` → `/cmd_vel`
→ `go2_nav2_bridge` → Go2 SDK

这时 Nav2 必须自己判断障碍和可通行空间，所以仍然绕不开：

- footprint 是否准确；
- 点云是否正确落在 `base_link/odom`；
- obstacle range 和 raytrace range；
- inflation radius；
- DWB/RPP 控制器参数；
- recovery 行为是否适合机器狗。

这和当前项目想要的“尽量少调 Nav2 复杂避障参数，主要依靠 Go2 官方近距离避障”不一致。

6.3 为什么当前选择自研 follower

当前自研 follower 很小，只做一件事：根据 odom 追踪相对 waypoint，然后发速度。它的好处是：

- 路线含义清楚：相对起点坐标；
- 行为可预测：按顺序到点、转向、继续；
- 调参少：主要调速度上限、比例增益、到点容差；
- 和 RSSI/IMU 实验匹配：路线复现比复杂全自主导航更重要；
- 不和 Unitree 官方避障抢同一层职责。

6.4 这不是否定 Nav2

当前方案不是说 Nav2 不好，而是分工不同。Nav2 更适合承担完整自主导航：建 costmap、规划路径、局部避障、恢复行为。当前 RSSI/IMU 项目阶段更需要一个可靠、简单、可解释的路线采集系统。

因此更准确的定位是：

当前项目使用 ROS 2/Nav2 兼容接口组织机器人状态和速度命令，但在线 waypoint 避障主链路采用自研 follower 加 Unitree 官方避障执行层，而不是 Nav2 官方 waypoint follower 加完整 Nav2 controller。

第七章 完整 Nav2 方案什么时候值得切换

7.1 适合切换的场景

如果后续目标从“RSSI/IMU 路线采集”变成“复杂环境自主导航”，完整 Nav2 方案就更有价值。例如：

- 需要绕过大障碍，而不是只做近距离防撞；
- 需要自动选择左右绕行；
- 需要动态重规划；
- 需要在论文中解释点云如何进入 costmap、如何影响速度评分；
- 需要复用 Nav2 的行为树、恢复行为、任务插件。

7.2 完整 Nav2 方案的链路

未来可以改成：

```
waypoints → nav2_waypoint_follower → bt_navigator → planner_server
           → controller_server → velocity_smoother → collision_monitor
           → go2_nav2_bridge → Unitree SDK
```

此时 /go2/pointcloud 会进入 VoxelLayer：

```
/go2/pointcloud → VoxelLayer → local costmap
                  → DWB/RPP → /cmd_vel
```

但这也意味着必须投入时间调 costmap、footprint、obstacle range 和 controller 参数。

7.3 两种方案对比

维度	当前方案	完整 Nav2 waypoint 方案
路线输入	relative waypoint list	Nav2 waypoint action/list
速度生成	自研 follower	Nav2 controller, 如 DW-B/RPP
近距离避障	Unitree obstacles_avoid	Nav2 costmap/controller + 可选 Unitree 兜底
参数复杂度	低	高
可解释性	控制逻辑清楚, 避障执行黑盒	Nav2 避障链路可解释, 但配置复杂
适合阶段	RSSI/IMU 采集、路线复现	自主导航、复杂绕障、论文级 Nav2 分析

第八章 如何运行和观察当前算法

8.1 编译

```
cd ~/桌面/RSSI/RSSI/code/ros2_ws
colcon build --packages-select go2_nav2_bridge
source install/setup.bash
```

8.2 运行单段路线

```
ros2 launch go2_nav2_bridge go2_waypoint_obstacle_avoid.launch.py \
  network_interface:=eth0 \
  relative_waypoints:="[3.0, 0.0, 0.0]"
```

8.3 运行 L 型路线

```
ros2 launch go2_nav2_bridge go2_waypoint_obstacle_avoid.launch.py \
  network_interface:=eth0 \
  relative_waypoints:="[3.0, 0.0, 1.57, 3.0, 2.0, 1.57]"
```

8.4 观察话题

```
ros2 node list
ros2 topic hz /odom
ros2 topic echo /cmd_vel
ros2 topic hz /go2/pointcloud
ros2 topic echo /go2/path_s
ros2 run tf2_ros tf2_echo odom base_link
```

正常情况下应看到：

- `/go2_nav2_bridge`;
- `/go2_waypoint_follower`;
- `/odom` 持续更新;
- `/cmd_vel` 在未到达目标前持续有速度;
- `/go2/pointcloud` 按一定频率更新;
- `bridge` 日志中出现 `cmd_vel -> obstacles_avoid Move`。

第九章 调参指南

9.1 路线跟踪参数

参数	当前默认值	含义
<code>controller_frequency_hz</code>	20.0	follower 控制频率。
<code>max_vx_mps</code>	0.30	最大前向速度。
<code>max_vy_mps</code>	0.08	最大横向速度。
<code>max_wz_radps</code>	0.45	最大角速度。
<code>min_vx_mps</code>	0.10	非零前进时的最低速度。
<code>k_xy</code>	0.45	距离误差到平移速度的比例增益。
<code>k_yaw</code>	0.90	角度误差到角速度的比例增益。
<code>xy_tolerance_m</code>	0.20	路点位置容差。
<code>yaw_tolerance_rad</code>	0.30	路点航向容差。

表 9.1: waypoint follower 的主要路线跟踪参数。

9.2 安全参数

9.3 问题排查

现象	检查方向
机器人不动	检查 <code>/odom</code> 是否有数据、 <code>/cmd_vel</code> 是否发布、bridge 是否使用 <code>obstacles_avoid</code> 、SDK 网卡是否正确。
刚启动就停	多半是点云新鲜度门控触发，检查 <code>/go2/pointcloud</code> 频率。
拐点处抖动	减小 <code>k_yaw</code> 或增大 <code>yaw_tolerance_rad</code> 。

目标附近冲过头	减小 <code>min_vx_mps</code> 或 <code>k_xy</code> , 增大 <code>xy_tolerance_m</code> 。
路线偏离理想轨迹	可能是 odom 漂移、Unitree 避障主动绕开、地面打滑或起点 yaw 记录不稳。

参数	调参建议
<code>cmd_timeout_sec</code>	上游速度断流后的停止时间。过大不安全，过小可能误停。
<code>require_fresh_point_cloud_for_motion</code>	真实机器人建议保持 <code>true</code> 。
<code>point_cloud_motion_timeout_sec</code>	点云超时阈值。若点云频率低，可略微增大，但不要无限放宽。
<code>self_filter_boxes</code>	过滤机身附近点。过大可能删掉近距离墙面，过小可能保留自车点。
<code>max_vx_mps,</code> <code>max_vy_mps,</code> <code>max_wz_radps</code>	<code>bridge</code> 最终限速，应不大于 <code>follower</code> 侧或与其一致。

表 9.2: `bridge` 的主要安全参数。